

# FLIPKART LOGISTICS

## OPTIMIZATION FOR DELIVERY ROUTES



*SQL-driven analytics to reduce delays, optimize routes & improve agent efficiency*



**Dataset:** 300 Orders | 20 Routes | 10 Warehouses |  
50 Agents | 999 Checkpoints

**Tool:** MySQL Workbench | **Tasks:** Data Cleaning · Delay  
Analysis · Route Optimization · Warehouse · Agent · KPI  
Reporting

### **VIDEOLINK**

[https://drive.google.com/file/d/1BK4eKtBt70Aiy\\_qqoZ4twGbNq3ZKMtmY/view?usp=sharing](https://drive.google.com/file/d/1BK4eKtBt70Aiy_qqoZ4twGbNq3ZKMtmY/view?usp=sharing)

BY- KAMLESH KUMAR  
THAKUR

# PROJECT OVERVIEW

## Challenge

Rising order volumes during festive seasons cause delays, route inefficiencies, and traffic disruptions affecting customer satisfaction.

## Approach

Leverage 5 relational tables (Orders, Routes, Warehouses, Agents, Tracking) with SQL queries, CTEs, window functions and aggregations.

## Objective

Build a SQL-driven analytics system to analyze delays, optimize routes, and enhance shipment & agent efficiency using data-driven insights.

## Outcome

Actionable recommendations on route optimization, warehouse bottlenecks, agent training and KPI dashboards for Ekart Logistics

# Data Cleaning & Preparation

- ▶ No duplicate records
- ▶ Converted data types
- ▶ Date format already standardized
- ▶ No null values in Traffic\_Delay\_Min
- ▶ No records found where Actual\_Delivery\_Date is before Order\_Date

The screenshot shows a database interface with a query editor and a result grid. The query editor contains the following SQL code:

```
26 SELECT *
27 FROM orders_table
28 WHERE Actual_Delivery_Date < Order_Date
```

The result grid below the query editor shows the following columns:

| Order_ID | Warehouse_ID | Route_ID | Agent_ID | Order_Date | Expected_Delivery_Date | Actual_Delivery_Date | Status | Order_Value |
|----------|--------------|----------|----------|------------|------------------------|----------------------|--------|-------------|
|----------|--------------|----------|----------|------------|------------------------|----------------------|--------|-------------|

## QUERIES USED

1. `SELECT Order_ID, COUNT(*) AS cnt  
FROM orders_table  
GROUP BY Order_ID  
HAVING COUNT(*) > 1;`
2. `ALTER TABLE orders_table  
MODIFY Order_Value DECIMAL(10,2);`
3. `SELECT * FROM orders_table  
WHERE Actual_Delivery_Date < Order_Date;`

# Task 2: Delivery Delay Calculation

Query-  
SELECT Order\_ID,  
Expected\_Delivery\_Date,  
Actual\_Delivery\_Date,  
DATEDIFF(Actual\_Delivery\_Date,  
Expected\_Delivery\_Date) AS Delay\_Days  
FROM orders\_table;

| Order_ID     | Expected_Delivery_Date | Actual_Delivery_Date | Delay_Days |
|--------------|------------------------|----------------------|------------|
| FLP-ORD-0001 | 2025-07-13             | 2025-07-14           | 1          |
| FLP-ORD-0002 | 2025-08-09             | 2025-08-09           | 0          |
| FLP-ORD-0003 | 2025-07-09             | 2025-07-09           | 0          |
| FLP-ORD-0004 | 2025-07-25             | 2025-07-25           | 0          |
| FLP-ORD-0005 | 2025-08-01             | 2025-08-01           | 0          |
| FLP-ORD-0006 | 2025-08-01             | 2025-08-02           | 1          |
| FLP-ORD-0007 | 2025-08-12             | 2025-08-12           | 0          |
| FLP-ORD-0008 | 2025-08-23             | 2025-08-23           | 0          |
| FLP-ORD-0009 | 2025-08-25             | 2025-08-25           | 0          |
| FLP-ORD-0010 | 2025-07-06             | 2025-07-09           | 3          |

| Order_ID     | Expected_Delivery_Date | Actual_Delivery_Date | Delay_Days |
|--------------|------------------------|----------------------|------------|
| FLP-ORD-0001 | 2025-07-13             | 2025-07-14           | 1          |
| FLP-ORD-0002 | 2025-08-09             | 2025-08-09           | 0          |
| FLP-ORD-0003 | 2025-07-09             | 2025-07-09           | 0          |
| FLP-ORD-0004 | 2025-07-25             | 2025-07-25           | 0          |
| FLP-ORD-0005 | 2025-08-01             | 2025-08-01           | 0          |
| FLP-ORD-0006 | 2025-08-01             | 2025-08-02           | 1          |
| FLP-ORD-0007 | 2025-08-12             | 2025-08-12           | 0          |
| FLP-ORD-0008 | 2025-08-23             | 2025-08-23           | 0          |

# Top Delayed Orders

Query-

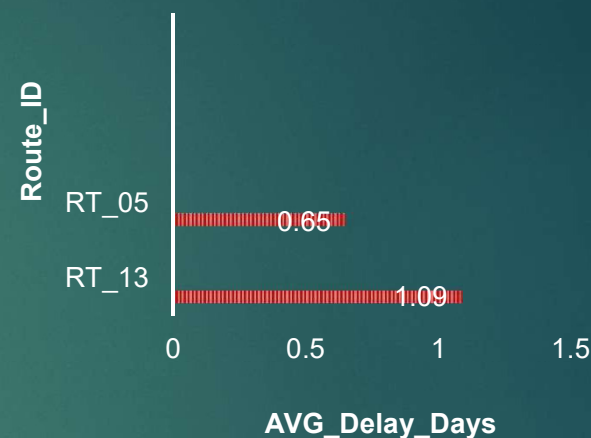
```
SELECT Order_ID,  
       DATEDIFF(Actual_Delivery_Date, Expected_Delivery_Date) AS  
       Delay_Days  
FROM orders_table  
ORDER BY Delay_Days DESC  
LIMIT 10;
```

| Order_ID     | Delays_days |
|--------------|-------------|
| FLP-ORD-0166 | 3           |
| FLP-ORD-0118 | 3           |
| FLP-ORD-0025 | 3           |
| FLP-ORD-0079 | 3           |
| FLP-ORD-0092 | 3           |
| FLP-ORD-0077 | 3           |
| FLP-ORD-0129 | 3           |
| FLP-ORD-0165 | 3           |
| FLP-ORD-0010 | 3           |
| FLP-ORD-0121 | 3           |

# Top Delayed Routes

```
SELECT Route_ID, AVG(DATEDIFF(Actual_Delivery_Date,  
Expected_Delivery_Date)) AS Avg_Delay_Days FROM orders_table  
GROUP BY Route_ID ORDER BY Avg_Delay_Days DESC LIMIT 10;
```

## DELAYED ROUTES



### KEY INSIGHTS:

Route RT\_13 was identified as the most delayed route with an average delay of 1.09 days.

Other routes such as RT\_05, RT\_14, and RT\_09 also showed moderate delays. This indicates that certain routes require operational improvements to reduce delivery time. These routes are inefficient and need optimization

-Targeted improvements in high-delay routes can significantly enhance overall delivery efficiency

- Higher Avg Delay = Poor route planning
- Lower Avg Delay = Efficient route

| Route_ID | Avg_Delay_Days |
|----------|----------------|
| RT_13    | 1.0909         |
| RT_05    | 0.6500         |
| RT_14    | 0.6471         |
| RT_09    | 0.6250         |
| RT_02    | 0.5882         |
| RT_18    | 0.5714         |
| RT_01    | 0.5385         |
| RT_12    | 0.5000         |
| RT_06    | 0.5000         |
| RT_16    | 0.5000         |



# Warehouse Performance Analysis

```
SELECT Warehouse_ID, Order_ID, DATEDIFF(Actual_Delivery_Date,
Expected_Delivery_Date) AS Delay_Days, RANK() OVER (
PARTITION BY Warehouse_ID ORDER BY
DATEDIFF(Actual_Delivery_Date, Expected_Delivery_Date) DESC)
AS Rank_in_Warehouse FROM orders_table;
```

| Warehouse | Order_ID     | Delay | Rank |
|-----------|--------------|-------|------|
| WH_01     | FLP-ORD-0077 | 3     | 1    |
| WH_01     | FLP-ORD-0194 | 3     | 1    |
| WH_01     | FLP-ORD-0210 | 2     | 3    |
| WH_01     | FLP-ORD-0055 | 2     | 3    |
| WH_01     | FLP-ORD-0148 | 1     | 5    |
| WH_01     | FLP-ORD-0296 | 1     | 5    |
| WH_01     | FLP-ORD-0002 | 0     | 7    |

Lowest delay = best warehouse

Orders ranked by delay within each warehouse

# Key Insights:

- Majority deliveries are on time (0 delay)
- Maximum delay observed: 3 days
- Certain routes (RT\_13) show higher delays
- Warehouse performance varies
- Improvement needed in specific routes like RT\_13.

## Conclusion:-

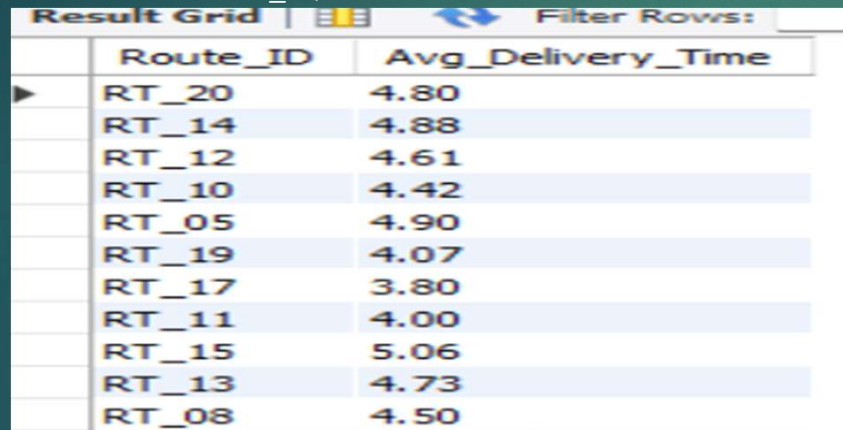
- Overall system performs well
- Delays are limited but concentrated
- Targeted improvements can enhance efficiency

# Task 3: Route Optimization Insights

- ▶ Route optimization is the process of finding the most efficient path or sequence of stops between multiple locations.
- ▶ The goal is to minimize the time, distance, cost or fuel use.

# Average Delivery Time Per Route

```
▶ SELECT
  Route_ID,
  ROUND(AVG(DATEDIFF(Actual_Delivery_Date, Order_Date)),2) AS Avg_Delivery_Time
FROM orders_table
GROUP BY Route_ID;
```



The screenshot shows a 'Result Grid' window with a table of average delivery times. The table has two columns: 'Route\_ID' and 'Avg\_Delivery\_Time'. The data is sorted by 'Avg\_Delivery\_Time' in descending order. The routes and their average delivery times are: RT\_20 (4.80), RT\_14 (4.88), RT\_12 (4.61), RT\_10 (4.42), RT\_05 (4.90), RT\_19 (4.07), RT\_17 (3.80), RT\_11 (4.00), RT\_15 (5.06), RT\_13 (4.73), and RT\_08 (4.50).

| Route_ID | Avg_Delivery_Time |
|----------|-------------------|
| RT_20    | 4.80              |
| RT_14    | 4.88              |
| RT_12    | 4.61              |
| RT_10    | 4.42              |
| RT_05    | 4.90              |
| RT_19    | 4.07              |
| RT_17    | 3.80              |
| RT_11    | 4.00              |
| RT_15    | 5.06              |
| RT_13    | 4.73              |
| RT_08    | 4.50              |

RT\_15 takes the longest with 5.06 days on average,  
followed by RT\_14 at 4.88 days and RT\_05 at 4.90 days.  
RT\_17 is the fastest with only 3.80 days.

# Average Traffic Delay

- ▶ `SELECT Route_ID,AVG(Traffic_Delay_Min) AS Avg_Traffic_Delay  
FROM routes_table GROUP BY route_id;`

| Route_ID | Avg_Traffic_Delay |
|----------|-------------------|
| RT_01    | 67.0000           |
| RT_02    | 30.0000           |
| RT_03    | 29.0000           |
| RT_04    | 23.0000           |
| RT_05    | 55.0000           |
| RT_06    | 15.0000           |
| RT_07    | 58.0000           |
| RT_08    | 90.0000           |
| RT_09    | 83.0000           |
| RT_10    | 15.0000           |
| RT_11    | 20.0000           |
| RT_12    | 58.0000           |
| RT_13    | 56.0000           |
| RT_14    | 36.0000           |

RT\_08 has the highest traffic delay at 90 minutes, followed by RT\_09 at 83 minutes and RT\_17 at 81 minutes. On the other end, RT\_06 and RT\_10 have the lowest traffic delay at just 15 minutes

# Efficiency Ratio

```
► SELECT  
r.Route_ID,r.Distance_KM,r.Average_Travel_Time_Min,ROUND(r.Distance  
_KM / r.Average_Travel_Time_Min, 2) AS Efficiency_RatioFROM  
routes_table r;
```

A higher ratio means the route covers more distance per minute of travel, which means it is more efficient

RT\_12 has the highest ratio of 4.98 and RT\_10 has 4.94 — these are the most efficient routes.

But RT\_13 has the lowest ratio of just 0.73 — meaning it covers very little distance relative to its travel time. RT\_14 is also poor at 1.00. These are the routes wasting the most time and need to be restructured or rerouted.

| Route_ID | Distance_KM | Average_Travel_Time_Min | Efficiency_Ratio |
|----------|-------------|-------------------------|------------------|
| RT_01    | 1009        | 631                     | 1.60             |
| RT_02    | 719         | 193                     | 3.73             |
| RT_03    | 846         | 749                     | 1.13             |
| RT_04    | 1713        | 869                     | 1.97             |
| RT_05    | 928         | 378                     | 2.46             |
| RT_06    | 1386        | 721                     | 1.92             |
| RT_07    | 1248        | 435                     | 2.87             |
| RT_08    | 1280        | 780                     | 1.64             |
| RT_09    | 1963        | 920                     | 2.13             |
| RT_10    | 1592        | 322                     | 4.94             |
| RT_11    | 1733        | 772                     | 2.24             |
| RT_12    | 1924        | 386                     | 4.98             |
| RT_13    | 1078        | 1481                    | 0.73             |
| RT_14    | 908         | 907                     | 1.00             |
| RT_15    | 1587        | 326                     | 4.87             |

# Worst 3 Routes (Efficiency)

- ▶ **SELECT**  
Route\_ID,Start\_Location,End\_Location,Distance\_KM,Average\_Travel\_Time\_Min,Traffic\_Delay\_Min,ROUND(Distance\_KM / Average\_Travel\_Time\_Min, 2) AS Efficiency\_Ratio FROM routes\_table  
ORDER BY Efficiency\_Ratio ASC LIMIT 3;

| Route_ID | Start_Location | End_Location | Distance_KM | Average_Travel_Time_Min | Traffic_Delay_Min | Efficiency_Ratio |
|----------|----------------|--------------|-------------|-------------------------|-------------------|------------------|
| RT_13    | Hyderabad      | Jaipur       | 1078        | 1481                    | 56                | 0.73             |
| RT_14    | Mumbai         | Mumbai       | 908         | 907                     | 36                | 1.00             |
| RT_03    | Hyderabad      | Mumbai       | 846         | 749                     | 29                | 1.13             |

Lower ratio = inefficient routes

# Routes with >20% Delayed Shipments

- SELECT Route\_ID, COUNT(\*) AS Total\_Orders, SUM(CASE WHEN Actual\_Delivery\_Date > Expected\_Delivery\_Date THEN 1 ELSE 0 END) AS Delayed\_Orders, ROUND(SUM(CASE WHEN Actual\_Delivery\_Date > Expected\_Delivery\_Date THEN 1 ELSE 0 END) \* 100.0 / COUNT(\*), 2) AS Delay\_Percentage FROM orders\_table GROUP BY Route\_ID HAVING SUM(CASE WHEN Actual\_Delivery\_Date > Expected\_Delivery\_Date THEN 1 ELSE 0 END) \* 100.0 / COUNT(\*) > 20 ORDER BY Delay\_Percentage DESC;

| Result Grid | Filter Rows: | Export:        | Wrap             |
|-------------|--------------|----------------|------------------|
| Route_ID    | Total_Orders | Delayed_Orders | Delay_Percentage |
| RT_13       | 11           | 6              | 54.55            |
| RT_05       | 20           | 8              | 40.00            |
| RT_14       | 17           | 6              | 35.29            |
| RT_17       | 15           | 5              | 33.33            |
| RT_06       | 12           | 4              | 33.33            |
| RT_09       | 16           | 5              | 31.25            |
| RT_01       | 13           | 4              | 30.77            |
| RT_16       | 10           | 3              | 30.00            |
| RT_02       | 17           | 5              | 29.41            |
| RT_18       | 14           | 4              | 28.57            |
| RT_12       | 18           | 5              | 27.78            |
| RT_15       | 17           | 4              | 23.53            |
| RT_07       | 9            | 2              | 22.22            |
| RT_10       | 19           | 4              | 21.05            |

## High-Risk Routes (Immediate Attention)

-These routes have very high delay %:

RT\_13 → 54.55% (worst)

RT\_05 → 40.00%

RT\_14 → 35.29%

More than 1 out of 3 deliveries are getting delayed.

-RT\_13 appears everywhere (Worst delay + Worst %).

# FINAL RECOMMENDATIONS

- ▶ Fix High-Risk Routes First:
- ▶ Focus on:
- ▶ RT\_13
- ▶ RT\_05
- ▶ RT\_14
- ▶ Review route planning
- ▶ Check traffic congestion patterns
- ▶ Reassign delivery agents if needed
- ▶ Investigate Root Cause
- ▶ Traffic issues

Improve Scheduling

Use Best Routes as Benchmark

# TASK:4~Warehouse Performance Analysis

## ► Top 3 Warehouses with Highest Processing Time

| Warehouse_ID | Warehouse_Name                        | City      | Average_Processing_Time_Min |
|--------------|---------------------------------------|-----------|-----------------------------|
| WH_10        | Flipkart Fulfillment Center Chennai   | Chennai   | 117                         |
| WH_09        | Flipkart Fulfillment Center Hyderabad | Hyderabad | 110                         |
| WH_01        | Flipkart Fulfillment Center Lucknow   | Lucknow   | 101                         |

- `SELECT Warehouse_ID, Warehouse_Name, City, Average_Processing_Time_Min  
FROM warehouse_table ORDER BY Average_Processing_Time_Min DESC LIMIT 3;`
- WH\_10 Chennai has the highest processing time at 117 minutes, followed by WH\_09 Hyderabad (110 min) and WH\_01 Lucknow (101 min). These 3 warehouses need operational improvements to reduce processing delays.

# Total vs Delayed Shipments per Warehouse

▶ `SELECT Warehouse_ID, COUNT(*) AS Total_Orders, SUM(CASE WHEN Actual_Delivery_Date > Expected_Delivery_Date THEN 1 ELSE 0 END) AS Delayed_Orders FROM orders_table GROUP BY Warehouse_ID;`

▶ **Worst Performing Warehouses**

-----WH\_10 → 11 delayed orders (highest)

-----WH\_08 → 10 delayed orders

-----WH\_02 & WH\_03 → 9 delayed orders

▶ These warehouses are major contributors to delivery delays and require operational improvements

▶ **Best Performing Warehouse**

▶ WH\_09 → 43 total orders but only 7 delayed

|   | Warehouse_ID | Total_Orders | Delayed_Orders |
|---|--------------|--------------|----------------|
| ▶ | WH_07        | 26           | 8              |
|   | WH_01        | 24           | 6              |
|   | WH_04        | 31           | 8              |
|   | WH_02        | 27           | 9              |
|   | WH_09        | 43           | 7              |
|   | WH_10        | 30           | 11             |
|   | WH_08        | 38           | 10             |
|   | WH_05        | 29           | 8              |
|   | WH_06        | 27           | 6              |
|   | WH_03        | 25           | 9              |

# Bottleneck warehouses (CTE)

- ▶ WITH avg\_time AS ( SELECT AVG(DATEDIFF(Actual\_Delivery\_Date, Order\_Date)) AS Global\_Avg FROM orders\_table)SELECT o.Warehouse\_ID,AVG(DATEDIFF(o.Actual\_Delivery\_Date, o.Order\_Date)) AS Avg\_Time FROM orders\_table o CROSS JOIN avg\_time a GROUP BY o.Warehouse\_ID HAVING AVG(DATEDIFF(o.Actual\_Delivery\_Date, o.Order\_Date)) > (SELECT Global\_Avg FROM avg\_time) ORDER BY Avg\_Time DESC;

| Warehouse_ID | Avg_Time |
|--------------|----------|
| WH_02        | 5.2963   |
| WH_10        | 4.8333   |
| WH_06        | 4.7407   |
| WH_05        | 4.6897   |

- **WH\_02** is the biggest bottleneck (highest delay)
- Followed by **WH\_10, WH\_06, WH\_05**
- These warehouses need:
  - Process optimization
  - Resource allocation
  - Capacity improvement

# Rank warehouses based on on-time delivery percentage

- ▶ `SELECT Warehouse_ID, ROUND(SUM(CASE WHEN Actual_Delivery_Date <= Expected_Delivery_Date THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS On_Time_Percentage, RANK() OVER (ORDER BY SUM(CASE WHEN Actual_Delivery_Date <= Expected_Delivery_Date THEN 1 ELSE 0 END) * 1.0 / COUNT(*) DESC) AS Rank_Position FROM orders_table GROUP BY Warehouse_ID;`

| Warehouse_ID | On_Time_Percentage | Rank_Position |
|--------------|--------------------|---------------|
| WH_09        | 83.72              | 1             |
| WH_06        | 77.78              | 2             |
| WH_01        | 75.00              | 3             |
| WH_04        | 74.19              | 4             |
| WH_08        | 73.68              | 5             |
| WH_05        | 72.41              | 6             |
| WH_07        | 69.23              | 7             |
| WH_02        | 66.67              | 8             |
| WH_03        | 64.00              | 9             |
| WH_10        | 63.33              | 10            |

**WH\_09** =Best performing warehouse

**WH\_10** =Worst performer

# KEY INSIGHTS ( Warehouse Performance)

- WH\_09 achieved the highest on-time delivery rate (83.72%)
- WH\_10 showed the lowest performance (63.33%)
- Bottleneck warehouses like WH\_02 and WH\_10 require optimization
- Some warehouses (e.g WH\_06) handle high processing time but still maintain good delivery performance
- Operational improvements are required in low-performing warehouses

# Task 5: Delivery Agent Performance

- ▶ Rank agents (per route) by on-time delivery percentage
- ▶ SELECT  
Agent\_ID,Agent\_Name,Route\_ID,Avg\_Speed\_KMPH,On\_Time\_Delivery\_Percentage,Experience\_Years,RANK() OVER (PARTITION BY Route\_ID ORDER BY  
On\_Time\_Delivery\_Percentage DESC) AS Rank\_In\_Route FROM deliveryagents\_table  
ORDER BY Route\_ID, Rank\_In\_Route;

| Agent_ID | Agent_Name   | Route_ID | Avg_Speed_KMPH | On_Time_Delivery_Percentage | Experience_Years | Rank_In_Route |
|----------|--------------|----------|----------------|-----------------------------|------------------|---------------|
| AG_049   | Kiran Reddy  | RT_01    | 37.3           | 97.2                        | 2.6              | 1             |
| AG_047   | Pooja Patel  | RT_01    | 48.2           | 81.2                        | 3.5              | 2             |
| AG_002   | Vikram Nair  | RT_01    | 48.6           | 73.2                        | 9                | 3             |
| AG_038   | Arun Reddy   | RT_02    | 35.1           | 91.6                        | 6.7              | 1             |
| AG_007   | Vikram Patel | RT_02    | 39.8           | 85.9                        | 1.7              | 2             |
| AG_020   | Kiran Kumar  | RT_02    | 48             | 85.1                        | 1.1              | 3             |
| AG_028   | Priya Nair   | RT_02    | 47.4           | 81.7                        | 6.3              | 4             |
| AG_026   | Kiran Patel  | RT_02    | 54.2           | 72.2                        | 2.7              | 5             |
| AG_013   | Rajesh Patel | RT_03    | 53.1           | 94.1                        | 1.6              | 1             |
| AG_001   | Arun Nair    | RT_03    | 42.7           | 86.7                        | 5.2              | 2             |
| AG_017   | Rajesh Gupta | RT_04    | 36.2           | 90.4                        | 8.2              | 1             |
| AG_050   | Vikram Kumar | RT_04    | 46.9           | 79.7                        | 8.1              | 2             |
| AG_040   | Anita Gupta  | RT_05    | 45.9           | 93.4                        | 6.2              | 1             |
| AG_041   | Amit Sharma  | RT_05    | 35.8           | 80.2                        | 9.4              | 2             |
| AG_021   | Anita Patel  | RT_06    | 43.1           | 83.5                        | 8.8              | 1             |

# Agents with on-time % < 80%

- SELECT Agent\_ID, Agent\_Name, Route\_ID, Avg\_Speed\_KMPH, On\_Time\_Delivery\_Percentage, Experience\_Years, CASE WHEN On\_Time\_Delivery\_Percentage < 73 THEN 'Critical – Below 73%' WHEN On\_Time\_Delivery\_Percentage < 77 THEN 'Poor – 73% to 77%' ELSE 'Below Average – 77% to 80%' END AS Performance\_Level FROM deliveryagents\_table WHERE On\_Time\_Delivery\_Percentage < 80 ORDER BY On\_Time\_Delivery\_Percentage ASC;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

|   | Agent_ID | Agent_Name    | Route_ID | Avg_Speed_KMPH | On_Time_Delivery_Percentage | Experience_Years | Performance_Level    |
|---|----------|---------------|----------|----------------|-----------------------------|------------------|----------------------|
| ▶ | AG_006   | Kiran Kumar   | RT_20    | 41.5           | 70.5                        | 9.4              | Critical – Below 73% |
|   | AG_019   | Vikram Sharma | RT_08    | 52.5           | 72.1                        | 8.3              | Critical – Below 73% |
|   | AG_026   | Kiran Patel   | RT_02    | 54.2           | 72.2                        | 2.7              | Critical – Below 73% |
|   | AG_004   | Vikram Nair   | RT_06    | 45.2           | 73                          | 6.6              | Poor – 73% to 77%    |
|   | AG_035   | Anita Patel   | RT_09    | 37.3           | 73                          | 6                | Poor – 73% to 77%    |
|   | AG_002   | Vikram Nair   | RT_01    | 48.6           | 73.2                        | 9                | Poor – 73% to 77%    |
|   | AG_008   | Meena Kumar   | RT_18    | 50.3           | 73.6                        | 5.3              | Poor – 73% to 77%    |
|   | AG_014   | Pooja Reddy   | RT_16    | 39.3           | 73.7                        | 9.4              | Poor – 73% to 77%    |
|   | AG_012   | Rajesh Nair   | RT_18    | 39.6           | 76.2                        | 1.6              | Poor – 73% to 77%    |
|   | AG_011   | Sneha Sharma  | RT_17    | 44             | 76.9                        | 1.6              | Poor – 73% to 77%    |
|   | AG_036   | Vikram Reddy  | RT_20    | 39.2           | 79.6                        | 7.2              | Below Average – 7... |
|   | AG_050   | Vikram Kumar  | RT_04    | 46.9           | 79.7                        | 8.1              | Below Average – 7... |
|   | AG_032   | Pooja Nair    | RT_09    | 37.6           | 79.8                        | 1.6              | Below Average – 7... |

# avg speed of Top 5 vs Bottom 5 agents using subqueries

- ▶ `SELECT 'Top 5 Agents (Best On-Time %)' AS Agent_Group, ROUND(AVG(Avg_Speed_KMPH), 2) AS Avg_Speed_KMPH FROM (SELECT Avg_Speed_KMPH FROM deliveryagents_table ORDER BY On_Time_Delivery_Percentage DESC LIMIT 5) top5 UNION ALL SELECT 'Bottom 5 Agents (Worst On-Time %)' AS Agent_Group, ROUND(AVG(Avg_Speed_KMPH), 2) AS Avg_Speed_KMPH FROM (SELECT Avg_Speed_KMPH FROM deliveryagents_table ORDER BY On_Time_Delivery_Percentage ASC LIMIT 5) bottom5;`

|   | Agent_Group                       | Avg_Speed_KMPH |
|---|-----------------------------------|----------------|
| ▶ | Top 5 Agents (Best On-Time %)     | 44.08          |
|   | Bottom 5 Agents (Worst On-Time %) | 46.14          |

**KEY INSIGHTS-** Bottom agents drive **FASTER** (46.14 kmph) than top agents (44.08 kmph). Speed is NOT the problem — they are rushing deliveries and skipping proper handover steps. The solution is accuracy training, not speed improvement

# Suggestions for Low Performers

- ▶ **1. Agents who are Critical (below 73%) with high experience (5+ years)** → AG\_006 (9.4 yrs), AG\_019 (8.3 yrs), AG\_004 (6.6 yrs), AG\_035 (6.0 yrs), AG\_014 (9.4 yrs) **Suggestion:** These agents are experienced but underperforming — reassign them to shorter or less congested routes (like RT\_10, RT\_11 which have 0% delay) instead of training.
- ▶ **2. Agents who are Critical with low experience (below 3 years)** → AG\_026 (2.7 yrs) **Suggestion:** Enroll in a structured delivery training program and pair with a top-performing agent on the same route before working independently.
- ▶ **3. Agents driving too fast (above 48 kmph) but delivering poorly** → AG\_019 (52.5 kmph), AG\_026 (54.2 kmph), AG\_008 (50.3 kmph), AG\_002 (48.6 kmph) **Suggestion:** Speed is not the problem — accuracy is. Enforce a maximum speed policy and focus training on proper handover, customer confirmation, and proof-of-delivery steps.
- ▶ **4. General workload balancing**
  - ▶ → RT\_18 has 2 low performers (AG\_008 and AG\_012) on the same route
  - ▶ **Suggestion:** Redistribute orders from overloaded routes to better-performing routes. Use the on-time % data monthly to rotate agents across routes and prevent burnout.

# Task 6: Shipment Tracking Analytics

- ▶ For each order, the last checkpoint and time.
- ▶ WITH RankedCheckpoints AS (SELECT Order\_ID,Checkpoint,Checkpoint\_Time,Delay\_Reason,Delay\_Minutes,ROW\_NUMBER() OVER (PARTITION BY Order\_ID ORDER BY Checkpoint\_Time DESC ) AS rn FROM shipmenttracking\_table)SELECT Order\_ID, Checkpoint AS Last\_Checkpoint, Checkpoint\_Time AS Last\_Checkpoint\_Time,Delay\_Reason,Delay\_Minutes FROM RankedCheckpoints WHERE rn = 1 ORDER BY Order\_ID;

| Order_ID     | Last_Checkpoint | Last_Checkpoint_Time | Delay_Reason | Delay_Minutes |
|--------------|-----------------|----------------------|--------------|---------------|
| FLP-ORD-0002 | Hub_4_Bengaluru | 2025-08-24 10:27:00  | None         | 0             |
| FLP-ORD-0003 | Hub_1_Mumbai    | 2025-08-28 16:54:00  | None         | 0             |
| FLP-ORD-0004 | Hub_5_Jaipur    | 2025-08-24 23:15:00  | None         | 0             |
| FLP-ORD-0005 | Hub_3_Pune      | 2025-08-22 11:50:00  | None         | 0             |
| FLP-ORD-0006 | Hub_2_Lucknow   | 2025-08-13 05:48:00  | Weather      | 67            |
| FLP-ORD-0007 | Hub_4_Hyderabad | 2025-08-03 03:43:00  | Weather      | 38            |
| FLP-ORD-0008 | Hub_3_Ahmedabad | 2025-07-13 00:43:00  | Traffic      | 103           |
| FLP-ORD-0009 | Hub_2_Jaipur    | 2025-08-12 13:08:00  | Traffic      | 92            |
| FLP-ORD-0010 | Hub_2_Bengaluru | 2025-08-04 00:11:00  | None         | 0             |
| FLP-ORD-0011 | Hub_1_Jaipur    | 2025-08-28 04:41:00  | Traffic      | 112           |

FLP-ORD-0002, the last checkpoint was Hub 4 Bengaluru on 24th August with no delay.

For FLP-ORD-0006, the last checkpoint was Hub 2 Lucknow with a Weather delay of 67 minutes.

# The most common delay reasons

- ▶ `SELECT Delay_Reason, COUNT(*) AS Occurrences, SUM(Delay_Minutes) AS Total_Delay_Minutes, ROUND(AVG(Delay_Minutes), 2) AS Avg_Delay_Per_Incident, RANK() OVER (ORDER BY COUNT(*) DESC) AS Frequency_Rank FROM shipmenttracking_table WHERE Delay_Reason IS NOT NULL AND UPPER(TRIM(Delay_Reason)) != 'NONE' GROUP BY Delay_Reason ORDER BY Occurrences DESC;`

| Delay_Reason    | Occurrences | Total_Delay_Minutes | Avg_Delay_Per_Incident | Frequency_Rank |
|-----------------|-------------|---------------------|------------------------|----------------|
| Traffic         | 387         | 26593               | 68.72                  | 1              |
| Weather         | 192         | 12236               | 63.73                  | 2              |
| Technical Issue | 107         | 6862                | 64.13                  | 3              |

**KEY INSIGHTS:-** Traffic is the #1 cause of delays — 387 occurrences adding up to 26,593 total minutes of delay. That's 443 hours lost just due to traffic. Real-time route rerouting during peak hours would significantly reduce this.

# orders with >2 delayed checkpoints

- ▶ SELECT Order\_ID,COUNT(\*) AS Total\_Checkpoints, SUM(CASE WHEN Delay\_Minutes > 0 THEN 1 ELSE 0 END) AS Delayed\_Checkpoints,SUM(Delay\_Minutes) AS Total\_Delay\_Minutes FROM shipmenttracking\_table GROUP BY Order\_ID HAVING SUM(CASE WHEN Delay\_Minutes > 0 THEN 1 ELSE 0 END) > 2 ORDER BY Delayed\_Checkpoints DESC;

| Result Grid  | Filter Rows:      | Export:             | Wrap Cell Content:  |
|--------------|-------------------|---------------------|---------------------|
| Order_ID     | Total_Checkpoints | Delayed_Checkpoints | Total_Delay_Minutes |
| FLP-ORD-0202 | 9                 | 8                   | 477                 |
| FLP-ORD-0061 | 7                 | 7                   | 458                 |
| FLP-ORD-0114 | 8                 | 7                   | 561                 |
| FLP-ORD-0251 | 9                 | 7                   | 409                 |
| FLP-ORD-0128 | 7                 | 7                   | 599                 |
| FLP-ORD-0229 | 7                 | 7                   | 413                 |
| FLP-ORD-0026 | 6                 | 6                   | 464                 |
| FLP-ORD-0177 | 6                 | 6                   | 305                 |
| FLP-ORD-0067 | 6                 | 6                   | 388                 |
| FLP-ORD-0075 | 7                 | 6                   | 286                 |

# Task 7: Advanced KPI Reporting

- ▶ Average Delivery Delay per Region (Start\_Location)
- ▶ 

```
SELECT r.Start_Location AS Region, COUNT(o.Order_ID) AS
Total_Orders, ROUND(AVG(DATEDIFF(o.Actual_Delivery_Date,
o.Expected_Delivery_Date)), 2) AS Avg_Delay_Days, SUM(CASE WHEN
DATEDIFF(o.Actual_Delivery_Date, o.Expected_Delivery_Date) > 0 THEN 1 ELSE 0 END)
AS Delayed_Orders, ROUND(100.0 * SUM(CASE WHEN
DATEDIFF(o.Actual_Delivery_Date, o.Expected_Delivery_Date) <= 0 THEN 1 ELSE 0 END) /
COUNT(o.Order_ID), 2) AS OnTime_Pct FROM orders_table o JOIN routes_table r ON
o.Route_ID = r.Route_ID GROUP BY r.Start_Location ORDER BY Avg_Delay_Days DESC;
```

Ahmedabad is the worst performing region with an average delay of 0.61 days and only 69.70% on-time rate, out of 33 orders. Pune is second worst at 0.56 days.

Hyderabad is the best performing region with just 0.38 days average delay and 76.13% on-time rate — and it also has the highest volume with 155 orders, which makes it even more impressive

| Region    | Total_Orders | Avg_Delay_Days | Delayed_Orders | OnTime_Pct |
|-----------|--------------|----------------|----------------|------------|
| Ahmedabad | 33           | 0.61           | 10             | 69.70      |
| Pune      | 36           | 0.56           | 11             | 69.44      |
| Lucknow   | 13           | 0.54           | 4              | 69.23      |
| Bengaluru | 12           | 0.50           | 4              | 66.67      |
| Mumbai    | 51           | 0.47           | 16             | 68.63      |
| Hyderabad | 155          | 0.38           | 37             | 76.13      |

# On-Time Delivery %

- ▶ 

```
SELECT COUNT(*) AS Total_Deliveries,SUM(CASE WHEN  
DATEDIFF(Actual_Delivery_Date,Expected_Delivery_Date) <= 0 THEN 1 ELSE 0 END) AS  
OnTime_Deliveries,SUM(CASE WHEN  
DATEDIFF(Actual_Delivery_Date,Expected_Delivery_Date) > 0 THEN 1 ELSE 0 END) AS  
Delayed_Deliveries,ROUND(100.0 * SUM(CASE WHEN  
DATEDIFF(Actual_Delivery_Date,Expected_Delivery_Date) <= 0 THEN 1 ELSE 0 END) /  
COUNT(*), 2) AS OnTime_Delivery_Pct FROM orders_table;
```

| Result Grid    Filter Rows: <input type="text"/>   Export:  Wrap Cell Content:  |                  |                   |                    |                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|-------------------|--------------------|---------------------|
|                                                                                                                                                                                                                                                                                                                                        | Total_Deliveries | OnTime_Deliveries | Delayed_Deliveries | OnTime_Delivery_Pct |
| ▶                                                                                                                                                                                                                                                                                                                                      | 300              | 218               | 82                 | 72.67               |

# Average Traffic Delay per Route

- ▶ SELECT Route\_ID, Start\_Location, End\_Location, Traffic\_Delay\_Min AS Avg\_Traffic\_Delay\_Min, RANK() OVER (ORDER BY Traffic\_Delay\_Min DESC) AS Delay\_Rank, CASE WHEN Traffic\_Delay\_Min >= 80 THEN 'High Traffic Impact' WHEN Traffic\_Delay\_Min >= 50 THEN 'Medium Traffic Impact' ELSE 'Low Traffic Impact' END AS Traffic\_Category FROM routes\_table ORDER BY Traffic\_Delay\_Min DESC;

Result Grid

  Filter Rows:

Export: 

Wrap Cell Content: 

|   | Route_ID | Start_Location | End_Location | Avg_Traffic_Delay_Min | Delay_Rank | Traffic_Category      |
|---|----------|----------------|--------------|-----------------------|------------|-----------------------|
| ▶ | RT_08    | Pune           | Pune         | 90                    | 1          | High Traffic Impact   |
|   | RT_15    | Hyderabad      | Jaipur       | 87                    | 2          | High Traffic Impact   |
|   | RT_09    | Ahmedabad      | Mumbai       | 83                    | 3          | High Traffic Impact   |
|   | RT_17    | Mumbai         | Lucknow      | 81                    | 4          | High Traffic Impact   |
|   | RT_01    | Lucknow        | Bengaluru    | 67                    | 5          | Medium Traffic Impact |
|   | RT_07    | Mumbai         | Lucknow      | 58                    | 6          | Medium Traffic Impact |
|   | RT_12    | Hyderabad      | Lucknow      | 58                    | 6          | Medium Traffic Impact |
|   | RT_13    | Hyderabad      | Jaipur       | 56                    | 8          | Medium Traffic Impact |
|   | RT_05    | Pune           | Pune         | 55                    | 9          | Medium Traffic Impact |
|   | RT_19    | Hyderabad      | Lucknow      | 50                    | 10         | Medium Traffic Impact |

RT\_08 from Pune to Pune has the highest traffic delay at 90 minutes, ranked number 1. RT\_15 from Hyderabad to Jaipur is second at 87 minutes. RT\_09 and RT\_17 also fall in the High Traffic Impact category at 83 and 81 minutes. Routes between 50 and 80 minutes are Medium Impact — and anything below 50 is Low Impact.

# Conclusion

- ▶ Overall on-time rate: 72.67% (12% below 85% industry benchmark)
- ▶ RT\_13 is the worst route — 54.55% delayed
- ▶ WH\_10 Chennai is the biggest bottleneck — 117 min processing time
- ▶ Traffic = #1 delay cause — 387 incidents, 443 hours lost
- ▶ 13 agents (26%) need performance improvement
- ▶ **Recommendation:**~The main recommendation is to prioritize RT\_13, reduce processing time in WH\_10, implement real-time traffic rerouting, and train the 13 underperforming agents.